



Plasma Simulations by Example

Lubos Brieda



CRC Press
Taylor & Francis Group

Plasma Simulations by Example



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Plasma Simulations by Example

Lubos Brieda



CRC Press

Taylor & Francis Group

Boca Raton London New York

CRC Press is an imprint of the
Taylor & Francis Group, an **informa** business

CRC Press
Taylor & Francis Group
6000 Broken Sound Parkway NW, Suite 300
Boca Raton, FL 33487-2742

© 2019 by Taylor & Francis Group, LLC
CRC Press is an imprint of Taylor & Francis Group, an Informa business

No claim to original U.S. Government works

Printed on acid-free paper

International Standard Book Number-13: 978-1-138-34232-3 (Hardback)

This book contains information obtained from authentic and highly regarded sources. Reasonable efforts have been made to publish reliable data and information, but the author and publisher cannot assume responsibility for the validity of all materials or the consequences of their use. The authors and publishers have attempted to trace the copyright holders of all material reproduced in this publication and apologize to copyright holders if permission to publish in this form has not been obtained. If any copyright material has not been acknowledged please write and let us know so we may rectify in any future reprint.

Except as permitted under U.S. Copyright Law, no part of this book may be reprinted, reproduced, transmitted, or utilized in any form by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying, microfilming, and recording, or in any information storage or retrieval system, without written permission from the publishers.

For permission to photocopy or use material electronically from this work, please access www.copyright.com (<http://www.copyright.com/>) or contact the Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC is a not-for-profit organization that provides licenses and registration for a variety of users. For organizations that have been granted a photocopy license by the CCC, a separate system of payment has been arranged.

Trademark Notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

Visit the Taylor & Francis Web site at
<http://www.taylorandfrancis.com>

and the CRC Press Web site at
<http://www.crcpress.com>

To Sandra and April.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

Preface	xvii
---------	------

CHAPTER 1 ■ Fundamentals	1
--------------------------	---

1.1	INTRODUCTION	1
1.2	GAS SIMULATION APPROACHES	1
1.2.1	Direct Method	2
1.2.2	Boltzmann Equation	3
1.2.3	Maxwell-Boltzmann Distribution Function	5
1.2.4	Fluid and Kinetic Methods	6
1.2.5	Lagrangian and Eulerian Descriptions	8
1.2.6	Comparison of Fluid and Kinetic Approaches	9
1.3	ELECTROSTATIC PARTICLE IN CELL METHOD	11
1.3.1	Lorentz Force	11
1.3.2	Maxwell's Equations	12
1.3.3	Poisson's Equation	13
1.3.4	Simulation Main Loop	13
1.4	SINGLE PARTICLE MOTION	14
1.4.1	Domain Discretization	15
1.4.2	Finite Difference Method	19
1.4.3	Potential Solver	20
1.4.3.1	Tridiagonal Algorithm	21
1.4.3.2	Jacobi Iteration	23
1.4.3.3	Gauss-Seidel Scheme	24
1.4.3.4	Successive Over Relaxation	24
1.4.3.5	Convergence Check	25
1.4.4	Electric Field	27
1.4.5	Particle Motion	29

1.4.5.1	Forward Euler Method	30
1.4.5.2	Backward Euler	31
1.4.5.3	Leapfrog Method	31
1.4.5.4	Alternative Approaches	33
1.4.6	Interpolation	34
1.4.7	Diagnostics	36
1.5	SUMMARY	41
CHAPTER	2 ■ Grounded Box	45
2.1	INTRODUCTION	45
2.2	SIMULATION SETUP	45
2.2.1	Pseudocode	46
2.3	WORLD OBJECT	47
2.3.1	Constructor	48
2.3.2	Declaration and Implementation	49
2.3.3	Multiple Files	50
2.3.4	Header Guards	52
2.3.5	Implementation	52
2.3.6	Mesh Resolution	53
2.4	FIELD OBJECT	54
2.4.1	Memory Allocation	55
2.4.2	Operator Overloading	56
2.4.3	Templates	58
2.4.4	Move and Copy Constructors	59
2.4.5	Additional Operators	60
2.4.6	Output	61
2.4.7	Three-Component Vectors	61
2.4.8	Adding Fields to World	63
2.4.9	Initial Output	63
2.4.10	Visualization	65
2.5	POTENTIAL SOLVER	66
2.6	ELECTRIC FIELD	70
2.7	PARTICLES	72
2.7.1	Particle Loading	73
2.7.2	Random Numbers	74

2.7.3	Node Volumes	77
2.7.4	Scatter: Particle to Mesh Interpolation	78
2.7.5	Number Density	81
2.8	CHARGE DENSITY	81
2.8.1	Numerical Noise	82
2.8.2	Quiet Start	83
2.9	PARTICLE PUSH	85
2.9.1	Gather: Mesh to Particle Interpolation	86
2.9.2	Velocity and Position Update	87
2.9.3	Particle Boundary Check	88
2.9.4	Velocity Rewind	89
2.9.5	Time Step	91
2.10	DIAGNOSTICS	92
2.10.1	Elapsed Time	94
2.10.2	Particle Counts, Momentum, and Energy	94
2.10.3	Animation	95
2.11	MAIN LOOP	96
2.12	RESULTS	97
2.12.1	Convergence Check	98
2.13	SUMMARY	98
CHAPTER	3 ■ Flow Around a Sphere	103
3.1	INTRODUCTION	103
3.2	SIMULATION DOMAIN	103
3.3	POTENTIAL SOLVER	105
3.3.1	Sugarcubing	105
3.3.2	Neumann Boundaries	107
3.3.3	Boltzmann Electron Model	108
3.3.4	Non-Linear Poisson Solver	109
3.4	PARTICLE INJECTION AND REMOVAL	112
3.4.1	Cold Beam Source	113
3.4.2	Particle Removal	115
3.4.3	Alternate Storage Schemes	117
3.5	STEADY STATE AND DATA AVERAGING	118
3.6	POTENTIAL SOLVER REVISITED	121

3.6.1	Matrix Representation	122
3.6.2	Sparse Matrix	124
3.6.3	Preconditioned Conjugate Gradient Solver	127
3.6.4	Newton-Raphson Linearization	129
3.6.5	Quasi-Neutral Approximation	132
3.7	SUMMARY	134
CHAPTER	4 ■ Material Interactions	137
4.1	INTRODUCTION	137
4.2	MULTIPLE SPECIES	137
4.3	SURFACE INTERACTIONS	138
4.3.1	Rebound Algorithm	138
4.3.2	Diffuse Reflection	141
4.3.3	Reflection Velocity	143
4.3.4	Species Change	144
4.3.5	Sputtering and Surface Emission	145
4.4	WARM BEAM	146
4.4.1	Velocity Sampling	149
4.4.2	Particle Merge	153
4.5	DIAGNOSTICS	154
4.5.1	Macroscopic Properties	154
4.5.2	Macroparticle Count	158
4.5.3	Particle Plots	159
4.6	COLLISIONS	162
4.6.1	Collision Cross Section	163
4.6.2	Collision Modeling	164
4.6.3	Chemistry	164
4.6.4	Monte Carlo Collisions (MCC)	166
4.7	DSMC	168
4.7.1	Particle Sorting	169
4.7.2	Collision Groups	169
4.7.3	Collision Probability	170
4.7.4	Momentum Exchange Collision Handler	171
4.8	SUBCYCLING	173
4.9	DIELECTRIC BOUNDARIES	174

4.10	DISCRETIZED SURFACES	176
4.10.1	Line Triangle Intersection	177
4.10.2	Particle Impacts	178
4.10.3	Caveats	179
4.11	SUMMARY	180
CHAPTER	5 ■ Symmetry	183
5.1	INTRODUCTION	183
5.2	SYMMETRY	183
5.2.1	Domain Boundary Condition	183
5.2.2	Particles	184
5.2.3	Results	185
5.3	PERIODIC SYSTEMS	185
5.3.1	Domain Boundary	186
5.3.2	Particles	188
5.3.3	Density Synchronization	188
5.3.4	Results	189
5.4	TWO-DIMENSIONAL CODES	189
5.5	PLANAR (XY) CODES	190
5.5.1	Node Volumes	191
5.5.2	Data Containers	192
5.5.3	Scatter and Gather	193
5.5.4	Output	194
5.6	AXISYMMETRIC (RZ) CODES	194
5.6.1	Node Volumes	194
5.6.2	Finite Volume Method	195
5.6.3	Particle Source	198
5.6.4	Scatter and Gather	199
5.6.5	Particle Rotation	200
5.6.6	Results	202
5.7	SUMMARY	202
CHAPTER	6 ■ Unstructured Meshes	205

6.1	INTRODUCTION	205
-----	--------------	-----

6.2	MESH GEOMETRY	205
6.2.1	Mesh Generation	206
6.2.2	Mesh Formats	207
6.2.3	Mesh Import	210
6.2.4	Cell Volumes	212
6.2.5	Field Object	212
6.2.6	Node Volumes	213
6.2.7	Output	213
6.3	PARTICLE INJECTION	215
6.4	MESH INTERPOLATION	217
6.4.1	Shape Factors	218
6.4.2	Cell Search	220
6.4.3	Cell Neighbors	222
6.4.4	Scatter and Gather	223
6.5	PARTICLE MOTION	224
6.5.1	Number Density	224
6.5.2	Neutral Results	225
6.6	FINITE ELEMENT METHOD	225
6.6.1	Strong Form	226
6.6.2	Weak Form	226
6.6.3	Galerkin Form	227
6.6.4	Formulation with Shape Functions	228
6.6.5	Matrix Form	228
6.6.6	Element View	230
6.6.7	Matrix Assembly	231
6.6.8	Quadrature	231
6.6.9	Shape Functions	233
6.6.10	Coordinate Transformation	234
6.6.11	Single Element Algorithm	235
6.6.12	Derivatives	236
6.6.13	Tetrahedral Element	236
6.7	PUTTING IT ALL TOGETHER	237
6.7.1	Initialization	237
6.7.2	Matrix Assembly	238
6.7.3	Shape Function Derivatives	240
6.7.4	Nonlinear Solver	242

6.7.5	Solution Assembly	244
6.7.6	Electric Field	245
6.7.7	Results	245
6.8	SUMMARY	245
CHAPTER	7 ■ Electromagnetics	249
7.1	INTRODUCTION	249
7.2	CLASSIFICATION	249
7.3	MAGNETIZATION	249
7.3.1	Boris Push	250
7.4	MAGNETOSTATICS	253
7.4.1	Vector Potential	254
7.4.2	Scalar Potential	254
7.4.3	Magnetized Sphere	255
7.5	ELECTROMAGNETIC PIC	258
7.5.1	Curl	259
7.5.2	Staggered Grid	260
7.5.3	Time Integration	261
7.5.4	Boundary Conditions	262
7.5.5	Gauss' Law	262
7.5.6	Putting It All Together	263
7.5.7	Approximations	264
7.5.8	Relativistic Push	264
7.5.9	Example	265
7.6	SUMMARY	269
CHAPTER	8 ■ Eulerian Methods	273
8.1	INTRODUCTION	273
8.2	MAGNETOHYDRODYNAMICS	273
8.2.1	Conservation Equations	273
8.2.2	Single Fluid MHD Equations	276
8.3	ADVECTION-DIFFUSION EQUATION	278
8.3.1	Steady State Form	279
8.3.2	Unsteady Problem	283

8.3.3	Stability Analysis	286
8.3.4	Alternative Time Integration Methods	287
8.3.5	Example	289
8.4	ELECTRON MODELS	293
8.4.1	Polytropic Relationship	294
8.4.2	Detailed Electron Model	295
8.5	VLASOV SOLVERS	296
8.5.1	Two-Stream Instability Example	297
8.6	SUMMARY	302
CHAPTER	9 ■ Parallel Programming	305
9.1	INTRODUCTION	305
9.2	ARCHITECTURES	305
9.2.1	Multithreading	307
9.2.2	MPI	308
9.2.3	CUDA	310
9.3	CAVEATS	311
9.3.1	Race Condition	311
9.3.2	Locks	312
9.3.3	Deadlock	313
9.3.4	Overhead	314
9.4	PROFILING	317
9.5	PARALLELIZATION STRATEGIES	318
9.5.1	Domain Decomposition	318
9.5.2	Ensemble Averaging	319
9.5.3	GPU Concurrent Computing	319
9.6	EXAMPLES	320
9.6.1	Multithreaded Particle Push	320
9.6.1.1	Number of Threads	321
9.6.1.2	Density Calculation	321
9.6.1.3	Parallel Efficiency	323
9.6.2	MPI Domain Decomposition	323
9.6.2.1	Subdomains	324
9.6.2.2	Particle Transfer	326
9.6.2.3	Boundary Update	329

9.6.2.4	Potential Solve	331
9.6.3	CUDA Potential Solve	333
9.7	SUMMARY	336
Bibliography		339
Index		345



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Preface

THE OBJECTIVE of this book is to teach you how to develop plasma simulation codes through easy to follow examples. While many excellent books already exist on the mathematical formulation of plasma simulation methods, they contain few, if any, practical code samples. As the saying goes, the devil is in the details. I have been developing plasma and rarefied gas codes for the past fifteen years. This book is my attempt to capture some of the intricacies that are encountered along the way. But, in order to keep the page count reasonable, it is not a detailed survey. Entire books have been written on topics covered in individual chapters. References included in the “Background” section include texts that I have found invaluable in my past work. These books contain discussions of alternate schemes or higher order methods that should be straightforward to implement once you grasp the basics covered in this text.

We focus on the kinetic Particle in Cell (PIC) method, although fluid methods are also discussed. The [first chapter](#) starts by introducing fundamental concepts, including the governing equations for the electrostatic Particle in Cell (ES-PIC) method. This is also where we discuss kinetic versus fluid modeling approaches. We then develop a simple particle integrator based on the Finite Difference Method to simulate an electron trapped in a potential well. The remainder of the book focuses on various forms of “a sphere in plasma” simulations. My background is in electric (plasma) propulsion, and the examples come from this research area. We mainly focus on low density discharges interacting with objects. The sphere is a placeholder that could represent a miniature Langmuir probe or a massive planet interacting with solar wind. In [Chapter 2](#), we put together the basic code framework and use it to perform a fully-kinetic simulation of ions and electrons interacting in a grounded box. We add the sphere in [Chapter 3](#). Here we also learn about hybrid approaches utilizing fluid electrons, and implement a faster potential solver based on the Preconditioned Conjugate Gradient (PCG) and Newton-Raphson (NR) methods. In [Chapter 4](#), we incorporate surface interactions, and also implement inter-particle collisions through the Monte Carlo Collisions (MCC) and Direct Simulation Monte Carlo (DSMC) methods. [Chapter 5](#) discusses methods for reducing simulation run times by taking advantage of problem symmetry. We develop two-dimensional planar and axisymmetric codes based on the Finite Volume Method. In [chapter 6](#), we modify the code to operate on an unstructured grid and learn about the Finite Element Method.

Electromagnetic effects are discussed in [Chapter 7](#). We cover the Boris method for integrating particle velocities, and compute the magnetic field for a magnetostatic problem. We also develop an electromagnetic PIC (EM-PIC) code. [Chapter 8](#) introduces Eulerian methods that can be used stand-alone, or can be coupled with the Lagrangian PIC method in a hybrid simulation. We also develop a 1D-1V solver for the Vlasov equation. Finally, we close the book in [Chapter 9](#) with a review of parallelization strategies. We discuss how to take advantage of multi-core CPU architectures using threads, and how to distribute simulations to multiple computers using MPI. We also cover basics of GPU programming using CUDA.

BACKGROUND

This book is written with the assumption that you are already familiar with basic plasma physics, linear algebra, and numerical analysis. If not, some useful references that I have encountered along the way include the works of Chen[24], Spitzer[52], Lieberman [42], Jackson[36], Sutton [54], Burden[23], Aris[10], and Boyce[17]. The mathematical formulation for kinetic and fluid plasma simulations is discussed in Birdsall [15], Tajima [55], Hockney [34], and Jardin [39]. Numerical methods for neutral gases are discussed by Anderson [8], Ferziger [28], Tu [56], and Bird [12]. The work of Hughes [35] forms the basis of the finite element solver in [Chapter 6](#). Finally, the books of Zhang[59], Jahn[37], and Hastings[33] provide an insight into the microscopic energy transfer, plasma propulsion, and the space environment.

All examples are developed in C++11, which is the 2011 revision of the C++ programming language. These days, Python and MATLAB® tend to be the primary languages taught in university settings. However, even with the help of the optimized Numpy libraries, Python remains many times slower than C++. Furthermore, the lack of variable types can lead to strange bugs, especially when a distinction between integer and floating point values is needed. Various scientific computing libraries are also implemented in C++, or interface with it without the need for external third party wrappers. These include the Message Passing Interface (MPI)[32] used for cluster computing and CUDA used for running code on NVIDIA graphics cards [58]. The downside of C++ is that it can be a bit overwhelming, especially if your background in objected oriented programming is limited. Furthermore, modern C++ introduces numerous new additions, such as standard library storage containers, loop iterators, and multi-threading, that you may not be familiar with if you have been mainly developing in the classic C. While the code is explained as we go along, I found the recent concise work of Stroustrup[53] to be an invaluable review of modern C++ language features.

CODE SAMPLES

Including in print the source code for all examples would be very much impractical and perhaps not very useful. The shorter codes in the first two chapters are replicated mainly in full, however, the rest of the book focuses only on the important pieces. The complete source code can be downloaded from the companion website

`https://www.particleincell.com/plasma-by-example`

There you will also find color versions of the plots in the book. The example codes were tested with the GNU g++ compiler version 7.4.0, running on Ubuntu 18 Linux. This version natively enables the C++11 language extensions. If you are using an older compiler, you may need to enable the `-std=c++11` flag, i.e.

```
g++ -std=c++11 *.cpp -o sphere
```

While C++ is a powerful language, it does not include any built-in graphical support. We output simulation results in the Visualization Toolkit (VTK) formats [51]. These files can then be visualized using programs such as Paraview or VisIt. These programs can be downloaded from <https://www.paraview.org/> and <https://wci.llnl.gov/simulation/computer-codes/visit/>

ACKNOWLEDGEMENTS

This book would not be possible if it were not for the many professors, research partners, and colleagues I have had the privilege of working with. I entered the world of numerical plasma physics by a chance when, as an undergraduate student at Virginia Tech, I joined the research group of a new professor Joseph Wang who was looking for students to help rewrite a legacy Fortran plasma plume code. Joining his group helped satisfy a technical elective requirement. Shortly afterwards, we were tasked with developing a new general electric propulsion modeling software based on a Cartesian mesh with cut cells. A team at MIT, led by Prof. Martinez-Sanchez and consisting of Mark Santi and Shannon Cheng, worked on a complimentary solver based on the unstructured mesh. I became the primary developer, but other students also made significant contributions. Raed Kafafy developed an immersed finite element solver, while Julien Pierru worked on a parallel version, and Alex Barrie added surface charging. Later, Randy Spicer performed extensive code validation against experimental data taken by another student from our group, Mike Nakles. After graduating I continued the code development at the Air Force Research Lab. There I collaborated with many brilliant scientists and engineers, including Mike Fife, Doug VanGilder, Justin Koo, Rob Martin, Jun Araki, David Bilyeu, Michelle Scharfe, John Loverich, and Michael Gorrilla. After three years, an opportunity arose to join professor Michael Keidar at

the George Washington University for a doctoral program. Some of the most memorable courses from my entire schooling career were taken here. Working with Prof. Keidar, I mainly focused on electron transport in Hall thrusters, but also briefly collaborated with his students Taisen Zhuang on vacuum arcs, and Olga Volotskova on atmospheric plasma jets for medical applications. Many fun moments were shared with my office mates Madhu Kundrapu and Brent Duffy.

While working on the doctorate, I also started a scientific computing blog at particleincell.com. Shortly after graduating, an idea arose to try offering an on-line plasma simulation course. I posted a note on the website, and to my surprise, almost twenty students signed up for the inaugural *Fundamentals of the PIC Method*. In the following years, I added a course on *Advanced PIC, Distributed Computing*, and *Fluid Modeling of Plasmas*. The feedback from the students was invaluable in shaping the material that eventually morphed into this book. Finally, much thanks goes to the book reviewers, to my parents and sister, and to my wife Sandra for putting up with the many late nights spent working first on the dissertation, and then later, this text.

Lubos Brieda
lubos.brieda@particleincell.com

Fundamentals

1.1 INTRODUCTION

THIS CHAPTER starts with a review of kinetic and fluid approaches for simulating plasma flows. The governing equations of the Electrostatic Particle in Cell (ES-PIC) method are then introduced. We next discuss the Finite Difference Method for discretizing differential equations. We use the method to simulate an electron trapped in a potential well.

1.2 GAS SIMULATION APPROACHES

Plasma, on the microscopic scale, is simply a gas containing charged particles: ions and electrons. That is at least the simplistic view we take in this book. More complex flows can be found in nature. Plasma may contain negative ions or large dust particulates that collect charge through triboelectric charging or photoemission. There may be multiphase flows involving gaseous components and liquid droplets. We do not consider such cases here, and instead focus solely on the basic combination of neutral atoms, positive ions, and enough electrons to neutralize the space charge. The objective of flow simulations is to predict the evolution of particle velocities and positions given some initial conditions and governing laws. Within the realms of Newtonian physics, we assume that all neutrals, ions, and electrons are rigid bodies governed by the equations of motions,

$$\frac{d\vec{x}}{dt} = \vec{v} \quad \frac{d\vec{v}}{dt} = \vec{a} \quad (1.1)$$

where $\vec{x}$, $\vec{v}$, and $\vec{a}$ are the position, velocity, and acceleration of a single particle. If the particle mass m remains constant, as is assumed throughout this book, the acceleration can be written in terms of total force, $\vec{a} = \sum \vec{F}/m$ per Newton's Second Law. Some forces, such as gravity, originate from the external environment. Other forces may be intrinsic to the system. Charged particles interact with each other through the *Coulomb force*,

$$\vec{F}_{c,ij} = \frac{1}{4\pi\epsilon_0} \frac{q_i q_j}{|\vec{x}_i - \vec{x}_j|^3} (\vec{x}_i - \vec{x}_j) \quad (1.2)$$

2 ■ Plasma Simulations by Example

where the i and j are indexes of two unique particles with charges q_i and q_j . $\epsilon_0 \approx 8.8542 \times 10^{-12}$ C/(Vm) is a physical constant known as *permittivity of free space*. This formulation ignores magnetic field effects found with moving charges. The total force acting on a single particle i is

$$\vec{F}_{c,i} = \sum_j^N \vec{F}_{c,ij} \quad i \neq j \quad (1.3)$$

where the sum is over all particles in the system. Particle velocity can also change through collisions. We thus have

$$\frac{d\vec{v}}{dt} = \frac{1}{m} \left(\vec{F}_g + \vec{F}_c + \dots \right) + \left(\frac{d\vec{v}}{dt} \right)_{col} \quad (1.4)$$

Although gravity is included in the above formulation, it is customary to ignore it. A quick “back of the envelope” calculation demonstrates why. Large vacuum chambers used in plasma processing operate at pressures p around 10^{-6} Torr (1 Torr is 1/760th of 1 atmosphere). Using the ideal gas law, $p = nk_B T$, where $k_B \approx 1.381 \times 10^{-23}$ J/K is the Boltzmann constant and T is the room temperature, the average number density n is around 3.3×10^{16} particles per cubic meter. The mean inter-particle spacing can be estimated from the volume needed to be assigned to each ion to fill a unit cube. Modeling each particle as a cube with sides a , the particles are $a = (1/n)^{1/3}$ or approximately $3 \mu\text{m}$ apart. At this distance, the Coulomb force is around 10^{-17} N. Even for the heavy xenon used in plasma propulsion, the force of gravity, $F = mg$, is only 10^{-24} N. The gravitational force is thus seven orders of magnitude smaller than the electrostatic Coulomb force. In the Low Earth Orbit (LEO), the ambient plasma density drops to 10^{12} m^{-3} , but the Coulomb force still dominates by four orders of magnitude. Therefore, unless we are dealing with problems involving massive gravitational fields, it is safe to ignore the force of gravity. Collisions can also be ignored at sufficiently low pressures.

1.2.1 Direct Method

Conceptually, we could model a system containing N particles by looping through the entire list, and for each, evaluating the Coulomb force from

$$\vec{F}_{c,i} = q_i \vec{E}_i \quad (1.5)$$

where

$$\vec{E}_i = \sum_j \frac{1}{4\pi\epsilon_0} \frac{q_j}{r_{ij}^3} (\vec{r}_i - \vec{r}_j) \quad i \neq j \quad (1.6)$$

This approach is inefficient. The Coulomb force needs to be evaluated $N - 1$ times for each particle. Since there are N particles in total, we need to perform $N(N - 1) \approx N^2$ Coulomb force calculations. It takes my computer 10^{-8} s to

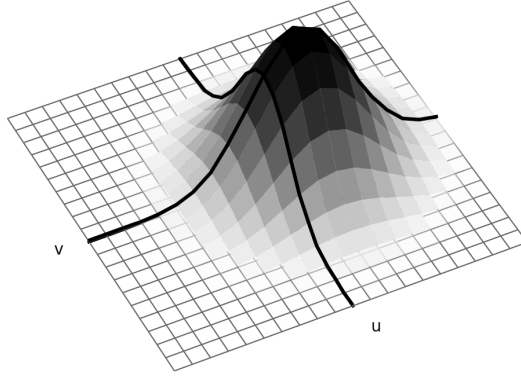


Figure 1.1: Example of a typical gas velocity distribution function.

evaluate Equation 1.2 for a single particle pair. Due to the N^2 dependence, it would take over 3×10^{17} years to compute $\vec{E}$ for all particles in just a single cubing meter at vacuum chamber pressures. This interval is seven orders of magnitude longer than the age of the universe! Clearly, this direct particle-particle approach is intractable. Fortunately, as we see shortly, there is a much faster way to compute the $\vec{E}$ term that scales with N .

Yet, even assuming the linear scaling, the computational run time would be 21 years due to the immense number of particles. Furthermore, even if some sudden breakthrough resulted in a massive increase in computational speeds, the amount of memory required to store the particle data is staggering. Over 700 billion Gb of RAM are needed to store just the six floating point numbers capturing particle position and velocity for the 10^{16} particles in a single cubic meter at vacuum pressures. This is beyond any supercomputer capacity.

1.2.2 Boltzmann Equation

Fortunately, alternatives exist. Imagine you could instantaneously freeze all particles within a small control volume for just long enough to group them by their velocity components. To accomplish the grouping, we define a histogram in the three-dimensional velocity space. Each bin corresponds to a unique combination of x , y , and z velocity components, u , v , and w . The width of each bin is given by Δu , Δv , and Δw . Particles with $u \pm 0.5\Delta u$, $v \pm 0.5\Delta v$, and $w \pm 0.5\Delta w$ all fall into the same bin. In the limit of the bins becoming infinitesimal, the discrete histogram becomes a smooth hyperspace surface. This surface represents the *velocity distribution function*, or VDF for short. [Figure 1.1](#) plots one possible VDF if only the u and v components are considered. The value of the function, represented by the height of the surface and the shading, indicates the number of particles with the corresponding u and v velocity components. We clearly see that the majority of particles are

4 ■ Plasma Simulations by Example

concentrated around some mean velocity pair. The two lines plot $u = 0$ and $v = 0$ that divide each velocity component into a negative and a positive half-space. The v velocities are centered on $v = 0$ and therefore there is no net motion in the y direction. On the other hand, there is a clear drift in x . Even though there are particles with $u < 0$, most of them have a positive u .

What makes this distribution-based approach powerful is realizing that, in the absence of collisions, all particles with the same velocity move in unison. This is a direct result of Newton's second law: $d\vec{v}/dt = \vec{F}/m$. Here we are assuming that the control volume is sufficiently small such that there is no spatial variation in the force term. There is no need to integrate the velocity for more than one particle per bin since the result is identical for all. It is only through collisions that some fraction of the population is affected in a unique way. We thus write a conservation statement

$$\frac{df}{dt} = \left(\frac{\partial f}{\partial t} \right)_{col} \quad (1.7)$$

This relationship indicates that the total derivative of the distribution function f changes only through some yet to be defined collision operator.

The distribution that we have discussed so far is valid only for the small control volume where it was sampled. Different volumes, corresponding to other spatial regions, have a unique combination of particle velocities. The distribution also evolves with time. The VDF is thus a function of seven variables, $f = f(x, y, z, u, v, w, t)$. Here f is the number of particles with velocity components (u, v, w) at position (x, y, z) and at time t . The distribution can be optionally normalized, in which case the function value $\hat{f}$ is the probability of finding such a particle. I prefer to work with the unnormalized version as it carries along all information needed to completely describe the gas state.

Utilizing chain rule to rewrite the derivative in Equation 1.7 leads to the *Boltzmann equation*,

$$\frac{\partial f}{\partial t} + \nabla \cdot f + \frac{F}{m} \nabla_v \cdot f = \left(\frac{\partial f}{\partial t} \right)_{col} \quad (1.8)$$

The collisionless variant, in which the right hand side is zero, is known as the *Vlasov equation*. Instead of considering inter-particle interactions directly, we can model gas evolution by integrating this partial differential equation (PDE). The vast majority of plasma simulation codes do not use this approach. The reason is simple and familiar: computational requirements. As was alluded previously, numerically storing an arbitrary VDF requires a three-dimensional velocity grid. This grid is basically a multi-dimensional array of floats, with the three indexes mapping to a particular u , v , and w bin. Even utilizing an extremely coarse $20 \times 20 \times 20$ resolution supporting only 20 discrete velocities per dimension, we need to store 8,000 floating point values per control volume. In order to resolve the spatial variation in gas properties, we also need to

discretize the physical domain into another three dimensional grid. Just as the velocity gridding limits the smallest discrete velocity difference, the volume grid controls the smallest distance that can be resolved. Utilizing a coarse $100 \times 100 \times 100$ discretization for the spatial domain, total of 800 million values need to be stored to capture a single VDF. This translates to almost 30 Gb of RAM. Integration schemes for the Boltzmann equation generally depend on at least one temporary buffer, doubling the memory requirements. Including additional gas species increases the storage correspondingly. While these requirements offer a huge improvement over the 700 billion Gb with the direct particle-particle scheme, they still greatly exceed the capabilities of standard computational systems. These requirements arise from the need to store six-dimensional data if all three spatial and velocity components are retained. At reduced dimensionality, direct solution of the Boltzmann equation can actually be quite attractive. We consider this class of solvers, confusingly called Vlasov solvers even when collisions are retained, in [Chapter 6](#).

1.2.3 Maxwell-Boltzmann Distribution Function

Clearly, alternatives must exist, since three dimensional plasma simulations are carried out routinely with much finer meshes on standard desktop workstations. Much of the memory requirement arises from the large number of velocity bins needed in each spatial cell. Most of these bins are effectively empty - this can in fact be seen in [Figure 1.1](#). Therefore, some additional simplifications can be made. The first option is to assume that the VDF follows an analytic profile. Specifically, we can let the velocity in each direction be given by

$$f_M(v) = \frac{1}{\sqrt{\pi}v_{th}} \exp\left(\frac{-(v - v_d)^2}{v_{th}^2}\right) \quad (1.9)$$

This very important equation is known as the *Maxwell-Boltzmann distribution function*. Quantum-theory or statistical mechanics can be used to derive that it is in fact the natural highest entropy state for a molecular population [59]. Imagine that you have a box initially filled with gas containing exactly two types of particles: slow molecules moving with speed A, and fast molecules moving with speed B. If there are no external forces, and the walls are perfectly specular, then velocities can change only through collisions. In the momentum exchange collision, the faster molecule slows down a bit while the slower one speeds up. The collision rate also scales inversely with the relative speed. Eventually, after a sufficient time has passed, the two initially discrete populations merge into a single one in which there is no net momentum transfer. The Maxwell-Boltzmann distribution corresponds to this end state and the process of reaching it is called *thermalization*.

The major benefit of utilizing Equation 1.9 is that it reduces the VDF to just two parameters for each dimension: the mean (or drift) velocity v_d , and the thermal velocity v_{th} . Thermal velocity is in turn defined in terms of temperature, $v_{th} = \sqrt{2k_B T/m}$. Therefore, just six floats fully describe the

VDF in each spatial cell. Often, we further assume isotropic temperature, reducing the memory required to just four values: the three components of drift velocity $\vec{v}_d$, and a single magnitude of thermal velocity. This is an immense improvement over the 8,000 values needed with the initial coarse histogram approach! But nothing in life is free, and this approach is not without a major drawback. In order to use Equation 1.9, we require that a “sufficient time” has passed for the gas to thermalize. This requirement is not always satisfied. The *collision rate* scales with number density, and the average distance a molecule travels between collisions is given by the *mean free path*,

$$\lambda = 1/(\sigma n) \quad (1.10)$$

The term σ is the *collision cross-section* and is discussed in more detail in [Chapter 4](#). n is the gas number density. An important factor, known as the *Knudsen number*, relates the mean free path to a characteristic length,

$$K_n = \frac{\lambda}{L} \quad (1.11)$$

This length is problem specific. In a simulation of a vacuum chamber, we can let L be the chamber diameter. If $K_n \ll 1$, implying $\lambda \ll L$, each molecule can be expected to undergo many collisions as it travels between the walls. Such a flow is said to be in *continuum*, and the Maxwellian VDF can be assumed to be valid. On the other hand, if $K_n \gg 1$, $\lambda \gg L$ and the molecules are much more likely to collide with the wall than each other. This state describes *free molecular flow*. Collisions can be completely ignored, and gas dynamics is driven by wall effects. Finally, if $K_n \approx 1$, the distance between collisions is comparable to the characteristic length. In this case, collisions happen but not frequently enough to assume the Maxwellian VDF with certainty. This state, known as *rarefied gas*, is often encountered in plasmas.

1.2.4 Fluid and Kinetic Methods

We can therefore divide gas simulation approaches into two categories: *fluid* and *kinetic*. Fluid approaches assume continuum with the VDF having an analytical form. Familiar gas properties such as density or mean velocity are obtained by evaluating *moments* of the distribution function. These are integrals of f convoluted with velocity raised to an increasingly higher power and evaluated over the entire velocity space,

$$M^k(\vec{x}, t) = \int_{\vec{v}} \vec{v}^k f(\vec{x}, \vec{v}, t) d\vec{v} \quad (1.12)$$

The zeroth, first, and second moments lead to conservation equations for mass, momentum, and energy. For the Maxwellian VDF, these moments give us the standard Navier-Stokes equations used in computational fluid dynamics (CFD).

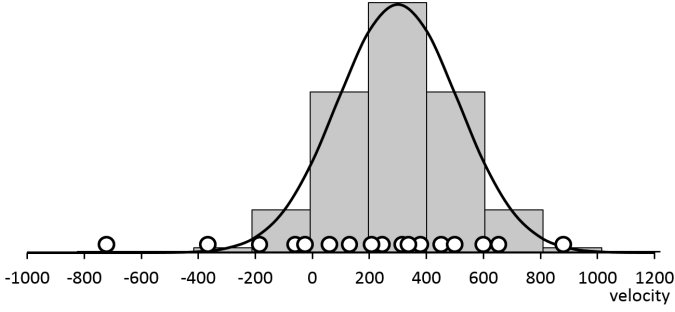


Figure 1.2: Comparison of three approaches for representing a velocity distribution function: analytical function (black line), a histogram of discrete velocities (gray bars), and random samples (white circles).

Kinetic approaches, on the other hand, do not make any assumptions about the shape of the VDF and allow it to evolve self-consistently. The direct and Vlasov solvers discussed previously are two examples of the kinetic approach. Both were found to have excessive computational requirements. Fortunately, there exists another class of methods based on *stochastic* (random) sampling. Many velocity bins contain zero or only a few particles. Instead of attempting to resolve the discretized VDF completely, we can generate a number of random velocity samples with probability following the distribution function. This sampling is illustrated in [Figure 1.2](#). The representation of the one-dimension Maxwell-Boltzmann distribution using the analytical shape is compared to a Vlasov solver velocity grid and the random sampling. We treat these samples as particles and let them move through the computational domain according to the equations of motion. In doing so, they convect their discrete velocity sample into new spatial regions. Each particle is assigned a *macroparticle weight* w_{mp} such that the initial number density is recovered using a much smaller number of samples. The macroparticle (or specific) weight is simply the number of real atoms, ions, or electrons represented by the simulation particle. For the case of constant weight, we have $w_{mp} = N_{real}/N_{sim}$. Two popular schemes based on this approach are the Particle in Cell (PIC) method [15] and Direct Simulation Monte Carlo (DSMC) [12]. PIC considers acceleration only due to electromagnetic forces. DSMC considers neutral gas and only models collisions. The two methods share many similarities and are often combined to model partially ionized gases. We develop a PIC-DSMC simulation in [Chapter 4](#). The stochastic nature introduces noise, which can be reduced by utilizing more samples (particles). The noise can also be reduced by averaging the results over a long interval once the solution stops evolving. Stochastic methods are thus best suited to problems in which the steady-state is achieved.

1.2.5 Lagrangian and Eulerian Descriptions

PIC and Vlasov solvers are both kinetic algorithms since they resolve the VDF self-consistently. They however treat the gas in a different way. While PIC follows individual particles as they move about the domain, Vlasov solvers evolve the gas using a stationary grid. These two models are known as *Lagrangian* and *Eulerian*. We can thus group gas simulation into four categories:

- Fluid/Eulerian: Computational Fluid Dynamics (CFD), MagnetoHydroDynamics (MHD)
- Fluid/Lagrangian: Smoothed Particle Hydrodynamics (SPH)
- Kinetic/Eulerian: Vlasov (direct kinetic) solvers
- Kinetic/Lagrangian: Particle in Cell (PIC), Direct Simulation Monte Carlo (DSMC)

The SPH scheme, which is not discussed in this text, is similar to PIC in that it represents the gas by Lagrangian particles. But unlike PIC, in which each particle carries a discrete velocity, SPH particles represent fluid parcels with a Maxwellian drift velocity and temperature. Finally, the PIC and DSMC schemes utilize a Eulerian spatial grid. This grid is used to compute inter-particle interactions. Recently, much progress has been made on grid-free particle methods that use dynamic octrees [26]. Such mesh-free approaches make the methods truly Lagrangian since the dependence on a spatial grid is eliminated.

The vast majority of this book is devoted to the kinetic PIC method. The reason is two fold. First, fluid methods share many similarities with their neutral gas counterparts. Instead of solving the Navier-Stokes equations as is done in computational fluid dynamics (CFD), the magnetohydrodynamics (MHD) equations are solved. The solution schemes are similar as they both involve numerically integrating systems of partial differential equations governing the conservation of mass, momentum, and energy. The main difference arises from the need to consider the evolution of the electromagnetic fields. Many excellent books already offer a comprehensive survey of relevant integration methods. But for completeness, fluid solvers are demonstrated in [Chapter 6](#). Fluid methods are also commonly coupled with PIC to form a hybrid scheme. In my field of electric propulsion, it is not common to simulate electrons directly. Instead, they are represented by fluid models of problem-specific complexity. Secondly, the PIC method is more versatile. The fluid methods are limited to the continuum regime. PIC, on the other hand, can be used across the entire range of Knudsen numbers, although they are best suited to rarefied and free molecular flows. The limitations of PIC arise from efficiency and not some inherent assumption about the VDF. Applying PIC to continuum flows requires much greater computational resources than the fluid approach, but in the end, the

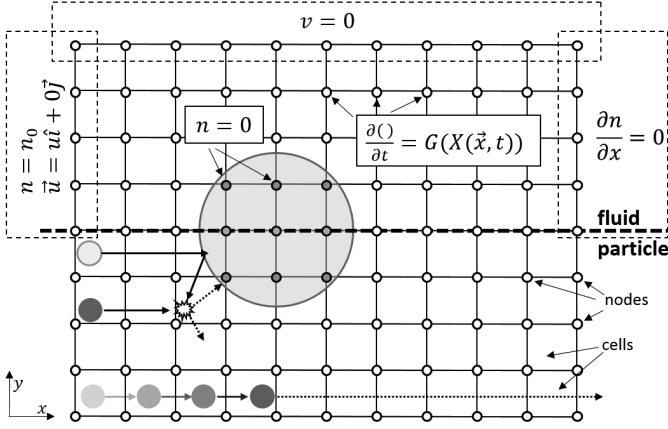


Figure 1.3: Comparison of a fluid (top) and particle (bottom) approach.

same result is obtained. The reverse is not true, as fluid approaches cannot model rarefied gases in which the velocity distribution is non-Maxwellian.

1.2.6 Comparison of Fluid and Kinetic Approaches

To illustrate the difference between fluid and particle methods, let's consider a simulation of neutral gas flow past a sphere. A schematic of this setup is shown in Figure 1.3. Starting with the fluid approach, we can use our understanding of gas dynamics (or look in appropriate reference books) to arrive at partial differential equations for the conservation of mass, momentum, and possibly energy. The energy equation is needed to compute temperature, but often we assume that temperature remains constant. For instance, we may have the following expressions for density and momentum [24]

$$\frac{\partial n}{\partial t} + \nabla \cdot (n\vec{u}) = 0 \quad (1.13)$$

$$mn \left[\frac{\partial \vec{v}}{\partial t} + (\vec{v} \cdot \nabla) \vec{v} \right] = \vec{F} - \nabla p - mn\nu\vec{v} \quad (1.14)$$

Details of these equations are not important right now. Instead we should notice that both contain a time derivative term $\partial()/\partial t$ and one or more terms that depend only on spatial properties (such as the $\vec{F}$ force term or the collisional drag term $mn\nu\vec{v}$) or their derivatives (the pressure gradient term ∇p). The equations can thus be recast into a form

$$\frac{\partial()}{\partial t} = G(X(\vec{x}, t)) \quad (1.15)$$

where G is some function of the spatially-varying gas state X (density, velocity, pressure, etc...) at time t . From this form we see that fluid simulations

involve time integration of differential equations for density, velocity, and so on. These governing equations do not have an analytical solution except in trivial cases, and some numerical scheme needs to be utilized. The scheme generally involves dividing the temporal domain into states separated by a small Δt time step. The physical domain is discretized into a spatial grid. This approach is needed, since the finite amount of computer memory does not make it possible to resolve the infinite amount of information available in the continuous real world. The integration then involves *time-marching* the solution from an *initial condition* until an exit criterion is met. This could involve simulating a prescribed real time, or the simulation reaching the *steady state* in which properties of interest no longer change, $\partial()/\partial t = 0$.

The blocks making up the spatial domain are called *cells*. The cell corners are known as *nodes* and are shared among neighbor cells. Cells can generally have an arbitrary shape but they should not overlap. The collection of cells is called a *simulation mesh* or a *simulation grid* if the cell ordering follows a logical structure. Figure 1.3 shows the uniform Cartesian grid with cube-shaped, constant-dimension cells and cell edges aligned with the three coordinate axes. This type of a mesh is ubiquitous in numerical simulations due to its simplicity. The simulation mesh makes it possible to define discrete points at which properties of interest, such as density or velocity, are known. The two obvious choices are the nodes and the cell centers. Unless noted otherwise, all simulations developed in this book utilize the *node-centered* approach. The governing equations are then cast to a form that makes it possible to evaluate them using the mesh-based data. We also specify *boundary conditions* that control how gas enters and leaves the computational domain. For instance, we may assign constant density $n = n_0$ and non-zero flow velocity in the x direction, $\vec{u} = u_0 \hat{i}$ on the x_{min} inlet plane. We also specify conditions on embedded boundaries. In Figure 1.3, these nodes are shown by the gray shading. As can be deduced from the plot, smooth geometries turn into degenerate *sugarcube* (or *stair case*) representations. This inability to resolve complex geometries is the primary limitation of Cartesian meshes. Some workarounds involve the use of adaptive mesh refinement [38] or cut-cells [40]. In Chapter 7 we see how to address this issue with the help of non-uniform meshes.

Loss of ions to surface neutralization can be modeled by setting ion density to zero, $n = 0$, on the nodes belonging to the sphere. This is an example of the *Dirichlet* boundary condition. This condition specifies the actual value for the unknown. It is not applicable everywhere. We do not know ahead of time the density on the downstream, $x = x_{max}$ face. Specifying the Dirichlet boundary condition there would be incorrect. Instead, we tell the code that the flow should be fully developed by requiring that there is no variation in the perpendicular (or normal) direction to the boundary, $\partial n / \partial \hat{n} \equiv \partial n / \partial x = 0$. This is an example of the *Neumann* boundary condition. Other types also exist, including Robin and Cauchy, but are encountered much less frequently.

The particle method, visualized in the bottom half, does not utilize any macroscopic governing equations at all. It instead simply injects particles into

the computational domain according to some source model. Here we inject particles on the x_{min} face. The number of injected particles is selected such that the desired density is achieved. The initial velocity is sampled from the inlet velocity distribution function. We advance particle velocities and positions by integrating the equations of motion, 1.1 through discrete Δt time steps. We first compute the new velocity of each particle by taking into account forces acting on it. We may also consider collisions. We then push particles to new positions by integrating velocity. Boundary conditions now control the microscopic behavior of individual particles. A neutral particle impacting the sphere may bounce off diffusely, while an ion may recombine into an atom. Particles leaving the computational domain may be removed or reflected back if the simulation models a problem exhibiting spatial symmetry. What makes particle simulations different from their fluid counterpart is that at each time step, the simulation only consists of a large collection of discrete position and velocities pairs. We are generally interested in computing the bulk densities, velocities, and temperatures. Obtaining these macroscopic properties from the microscopic particle data requires some additional averaging. This is generally accomplished with the help of a computational mesh. The local number density could be obtained by computing the number of particles in each cell since number density is just the particle count divided by the cell volume. Gas velocities and temperatures can be obtained in a similar manner.

1.3 ELECTROSTATIC PARTICLE IN CELL METHOD

With this in mind, we now introduce the Particle in Cell (PIC) method. This plasma simulation method was popularized by Birdsall and Langdon [15], and to this day, their book serves as an invaluable reference for anyone working in this field. Let's assume that some small region of space contains N real ions described by a local velocity distribution function f . We can approximate this population by sampling M simulation particles from the VDF. This much smaller sample represents the entire population, and hence each particle corresponds to $w_{mp} = N/M$ real ions or electrons. Each simulation particle is born with a discrete velocity and carries momentum $w_{mp}m\vec{v}$. Similarly, the particle contributes w_{mp} and $w_{mp}q$ count and charge to local number and charge density calculation. Velocity and position is updated according to the mass and charge of the individual physical ion or electron of mass m and charge q . In order to avoid confusion between the charge used to move the particle and the contribution to charge density, it is helpful to envision each macroparticle as a stack of w_{mp} real ions or electrons moving in unison.

1.3.1 Lorentz Force

Particles move according to the equations of motion discussed previously, with force computed per Equation 1.5. However, instead of using the Coulomb law, we obtain the $\vec{E}$ term, called the *electric field*, in a more efficient way. The

12 ■ Plasma Simulations by Example

relationship given by Equation 1.5 is actually incomplete as it neglects the effect of moving charges. The complete form is known as the *Lorentz force*,

$$\vec{F} = q \left(\vec{E} + \vec{v} \times \vec{B} \right) \quad (1.16)$$

where $\vec{v}$ is the particle velocity and $\vec{B}$ is the magnetic field.

1.3.2 Maxwell's Equations

The evolution of the electric and magnetic fields is given by the four fundamental equations of electromagnetics known as *Maxwell's equations*. They are:

$$\text{Gauss' law:} \quad \nabla \cdot \vec{E} = \frac{\rho}{\epsilon_0} \quad (1.17)$$

$$\text{Gauss' law for magnetism:} \quad \nabla \cdot \vec{B} = 0 \quad (1.18)$$

$$\text{Faraday's law:} \quad \nabla \times \vec{E} = -\frac{\partial \vec{B}}{\partial t} \quad (1.19)$$

$$\text{Ampere's law:} \quad \nabla \times \vec{B} = \mu_0 \left(\vec{j} + \epsilon_0 \frac{\partial \vec{E}}{\partial t} \right) \quad (1.20)$$

Here $\rho \equiv \sum_s q_s n_s$ is the *charge density*, and $\vec{j} \equiv \sum_s q_s n_s \vec{v}_s$ is the *current density*. Faraday's law tells us that if $\partial \vec{B} / \partial t = 0$, $\nabla \times \vec{E} = 0$. A vector field with a zero curl is said to be irrotational. Helmholtz decomposition, also known as the fundamental law of vector calculus, further tells us that any sufficiently smooth vector field $\vec{F}$ can be decomposed into an irrotational (curl-free) and a solenoidal (divergence-free) part. These two parts can be defined in terms of a scalar (for irrotational) and a vector (for solenoidal) potential,

$$\vec{F} = -\nabla \phi + \nabla \times \vec{A} \quad (1.21)$$

The scalar potential ϕ is included with a negative sign per convention. Therefore, if the magnetic field is time invariant, the solenoidal contribution must vanish, and the electric field can be defined in terms of scalar potential,

$$\vec{E} = -\nabla \phi \quad (1.22)$$

This $\partial \vec{B} / \partial t = 0$ condition is known as the *electrostatic assumption*. Since Ampere's law indicates that magnetic field arises from currents, this condition implies that current density is low enough for the self-induced magnetic field to be negligible. Furthermore, if external magnets are present, the imparted field cannot be varying with time.

1.3.3 Poisson's Equation

The electrostatic assumption vastly simplifies the remaining analysis. We can substitute Equation 1.22 into Gauss' law to obtain

$$\nabla \cdot (-\nabla \phi) = \frac{\rho}{\epsilon_0} \quad (1.23)$$

or

$$\nabla^2 \phi = -\frac{\rho}{\epsilon_0} \quad (1.24)$$

This is the Poisson's equation and it forms the basis of the Electrostatic Particle in Cell (ES-PIC) method. It provides a relationship between plasma potential and charge density. Charge density is simply the aggregate of number densities for individual species scaled by their charge. In a system containing only singly and doubly-charged ions and electrons, we have $\rho = e(n_i + 2n_{i2} - n_e)$, where e is the elementary charge. The density of the doubly charged ions can alternatively be written in terms of the singly-charged population, $n_{i2} = kn_i$, where k is some fraction. We then have $\rho = e[(1 + 2k)n_i - n_e] = e(Z_i n_i - n_e)$. The Z_i term is the average ionization state.

What makes this approach attractive is realizing that charge density is a macroscopic property. Practically this implies that we define density in terms of a simulation mesh. The number of mesh nodes G is many times smaller than the number of computational particles, M . Instead of the M^2 dependence required with the direct method, the operation count is reduced roughly to:

- M operations to compute charge density (loop over particles)
- $G \log(G)$ operations to solve Poisson's equation (loop over mesh nodes)
- G operations to compute electric field (loop over mesh nodes)
- M operations to integrate particle positions (loop over particles)

The $G \log(G)$ terms captures the typical convergence rate of solvers for the elliptic Poisson equation. Assuming we have a mesh with 100,000 cells and 20 particles per cell, the total operation count for the PIC scheme is reduced by six orders of magnitude compared to the direct Coulomb force summation. This is the great strength of the PIC method. It allows us to perform kinetic simulations in a manner that is compatible with standard desktop workstations.

1.3.4 Simulation Main Loop

In the PIC method, the computational domain is first discretized into a simulation mesh. At each time step, we use particle positions to compute the number density. We then use the corresponding charge density to compute plasma potential using Equation 1.24. Once the potential is known, the electric field is obtained from Equation 1.22. This electric field is used to update

14 ■ Plasma Simulations by Example

particle velocities according to the Lorentz force. The new velocity is used to push particles to new positions and the whole process repeats. The general ES-PIC algorithm follows the following steps:

```
// initialization
solve initial potential
compute electric field
load particles

// simulation loop
main_loop:
    // fields
    compute charge density
    compute potential
    compute electric field

    // particles
    update particle velocity
    update particle position

    // optional steps
    perform collisions
    inject additional particles
    output diagnostics

// finalization
output results
```

The simulation main loop continues until some exit criterion is met. This could involve completing a preset real time, or reaching the steady state. After the loop finishes, we typically save the final results and perform additional finalization. An electromagnetic PIC (EM-PIC), discussed in [Chapter 7](#), follows a similar outline, but a different method is used to advance the electric and magnetic fields.

1.4 SINGLE PARTICLE MOTION

We now show how the core concepts of the PIC method are actually implemented by developing a simple simulation program. Let's imagine that a region of uniform charge density ρ_0 exists between two grounded electrodes. This setup is shown in [Figure 1.4](#). We also assume that the electrodes are infinitely tall and deep. Since the charge density is uniform, plasma properties vary only in the x direction. This is an example of a one dimensional (1D) problem.

We assume that the region is filled only with ions, $\rho_0 = en_i$. For $\rho_0 > 0$, positive potential develops between the two walls. A single stationary electron placed near one of the two electrodes should start moving towards the top of the potential hill, where it reaches the maximum speed. It then continues down the hill, slowing down along the way until its velocity reaches zero again. The motion then reverses, and the electron continues to oscillate with this back and forth motion. Of course, this model ignores any frictional (or collisional)

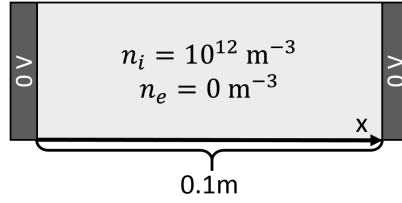


Figure 1.4: Simulation domain for the single particle motion example.

losses. In order to develop the simulation, we assume that the background ion density remains constant and that it is sufficiently high so the charge carried by the single electron has negligible effect on the potential profile. This then allows us to compute the potential just once. The rest of the code simply involves updating the electron position. We have the following pseudo-code:

```
solve potential
compute electric field
load particle

main_loop:
    update particle velocity
    update particle position
    write diagnostics
```

1.4.1 Domain Discretization

As was alluded to before, we use a computational mesh to describe the spatial variation in potential, charge density, and the electric field. Computationally this involves allocating a memory array to store n_i floating point values for each variable, where n_i is the number of mesh nodes. In classic C++, arrays can be dynamically allocated using the **new** operator,

```
// constants
namespace Const {
    const double QE = 1.602176565e-19; // C, elementary charge
};

int main() {
    const int ni = 21; // number of mesh nodes
    double *phi = new double[ni]; // potential
    double *rho = new double[ni]; // charge density
    double *ef = new double[ni]; // electric field

    // initialize the data
    for (int i=0; i<ni; i++) {
        phi[i] = 0;
        rho[i] = Const::QE*1e12; // charge density
        ef[i] = 0;
    }

    /* rest of the code goes here */
```


16 ■ Plasma Simulations by Example

```
    delete[] phi;                                // memory cleanup
    delete[] rho;
    delete[] ef;
    return 0;                                     // normal exit
}
```

This approach has several downsides. First, the allocated data is not initialized by default. Each array component initially contains just some random bytes that existed in the assigned memory location. We need to manually set each entry to zero (or other values that make sense) prior to use. Secondly, we need to remember to use the `delete[]` operator to release the memory once the array is no longer needed.

A cleaner approach is to use the `std::vector` container from the C++ standard library,

```
#include <vector>
using namespace std;           // allows skipping std:: in std::vector

int main() {
    const int ni = 21;
    vector<double> phi(ni);    // space for ni doubles, all set to 0
    vector<double> rho(ni, Const::QE*1e12); // set to QE*1e12
    vector<double> ef(ni);

    /* rest of the code goes here */

    return 0;                 // normal exit
}
```

You can notice two main differences: there is no longer the need to initialize the data. It is automatically set to zero, unless a different value is provided as is the case with charge density. We also don't need to free the memory - this happens automatically when the main block exits and the `vector` variables go out of scope. `std::vector` is a *templated* container capable of storing arbitrary data types. The `<double>` template argument tells the compiler that we need a container for double precision floating point values. Sometimes having to repeatedly type these arguments can become cumbersome. To get around this, we can define a new type called `dvector` with the help of the `using` directive,

```
#include <vector>
using namespace std;
using dvector = vector<double>;    // define a 'nickname'

int main() {
    const int ni = 21;
    dvector phi(ni); // ni doubles to hold potential

    /* rest of the code */

    return 0;           // normal exit
}
```